NEWBIT · PLATAFORMA DE COMPUTER VISION PARA OPERACIONES INDUSTRIALES

Pasos para desarrollar NewBit: de la cámara a la conducta segura

Doce pasos para construir la primera versión de NewBit: reconocer en video dos conductas inseguras (tránsito fuera de la vía peatonal y panel eléctrico abierto) y alertar a tiempo, con una persona que valida cada alerta.

← Índice · 1 · Doce pasos · 2 · Metodología

1 Definir el problema

Las inspecciones de seguridad son puntuales y las cámaras graban sin avisar: una conducta insegura dura segundos y se detecta tarde o nunca.



Ejemplo: un operario cruza fuera de la vía peatonal mientras pasa un montacargas. Nadie lo ve hasta revisar el video días después.

2 Definir objetivo y variables

Objetivo: construir NewBit para reconocer conductas inseguras en video industrial y alertar a tiempo.

Variable independiente: configuración del pipeline (selección de frames con YOLOv4 + clasificación temporal con ConvLSTM).

Variable dependiente: precisión por conducta y tiempo de procesamiento por clip.

Las metas numéricas se fijan con la línea base del Sprint 3.

3 Diseñar la experiencia de CV

Pipeline: qué ve y qué hace el usuario



Cámara → Detección → Secuencia → Alerta

Zona verde: vía peatonal configurada. La conducta se confirma a lo largo de la secuencia, no en un solo frame.

4 Seleccionar la tecnología

VIDEO	OpenCV · lectura de MP4 y frames
DETECCIÓN	YOLOv4 (Darknet)
MODELO TEMPORAL	ConvLSTM (PyTorch)
ANOTACIÓN	LabelImg
LENGUAJE	Python
BACKEND	FastAPI (propuesta)
BASE DE DATOS	PostgreSQL (propuesta)
INTERFAZ	Aplicación web con alertas
CÓMPUTO	GPU en Colab o local





5 Entrenar y configurar los modelos

YOLOv4 parte de pesos preentrenados (COCO) y se ajusta con frames anotados; el ConvLSTM aprende las conductas sobre secuencias de video.



Fotos de referencia. El entrenamiento usa un dataset público de video industrial (licencia CC BY 4.0) con su división oficial de entrenamiento y prueba.

6 Implementar el sistema NewBit

1. Ingerir el video (MP4).
2. Detectar personas y objetos por frame.
3. Seleccionar los frames relevantes.
4. Agrupar frames en secuencias.
5. Clasificar la conducta (ConvLSTM).
6. Generar la alerta con el clip.
7. Registrar y mostrar en la interfaz.

7 Incorporar la parte inteligente

Analítica de conductas

Conducta insegura · Confianza · Recurrencia por zona · Frames descartados

- Clase de conducta y nivel de confianza por secuencia.
- Zonas y horarios donde más se repite cada conducta.
- Reportes por tipo, zona y semana para la gerencia.

8 Diseñar la metodología de validación

Foco en el desempeño por conducta y en la eficiencia del pipeline.

Métricas por clase: precisión, recall, F1 y matriz de confusión.

Eficiencia: tiempo por clip y porcentaje de frames descartados.

Comparación: clasificación directa vs. pipeline completo, con la misma partición de prueba.

9 Validar la plataforma

Validación técnica
- Precisión y recall por conducta
- Tiempo de procesamiento
- Reproducibilidad

Validación experta
- Relevancia para SST
- Claridad de la alerta
- Falsas alarmas

La validación experta la hace el perfil de SST del caso académico en la revisión de cada sprint.

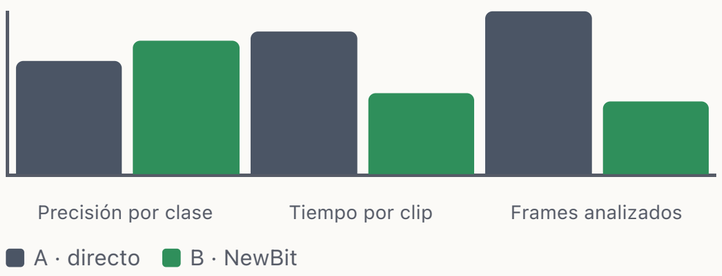
10 Realizar la evaluación comparativa

A · ConvLSTM directo (todos los frames) vs **B · NewBit** (YOLOv4 + ConvLSTM)

1. Medir la línea base (A) en el conjunto de prueba.
2. Aplicar el pipeline completo (B) a los mismos clips.
3. Comparar métricas por clase y tiempos.

11 Analizar resultados

Qué se compara entre A y B:



Barras ilustrativas: los valores reales salen de la evaluación.

12 Conclusiones

- **Efecto de la selección de frames** en precisión y velocidad.
- **Limitaciones:** datos de una planta y dos cámaras; video grabado.
- **Siguiente versión:** nuevas conductas y cámaras en vivo.
- **Buenas prácticas** para configurar umbrales y zonas.



Estructura completa del proyecto

4 sprints · Scrum

Problema · Objetivos · Diseño del pipeline · Anotación y entrenamiento · Backend e interfaz

Integración · Pruebas técnicas · Evaluación comparativa · Validación experta · Conclusiones

Tecnología (resumen)

OpenCV · YOLOv4 · Darknet · ConvLSTM · PyTorch · Python · LabelImg · FastAPI · PostgreSQL · Web

Equipo: Albert Chumacero (PO) · Anthony Lopez (SM y Dev) · Luis Salazar (Dev) · Ingeniería de Software II · UNS 2026-II

▶ Créditos de imágenes (Wikimedia Commons)